

# Turno

## Manual técnico

Instalação, conceitos, cada tela do painel, handlers, garantias de execução, resiliência, escala, API, configuração, testes e solução de problemas do motor de tarefas distribuídas em C# e .NET 10.

**Versão 1.7.0**
Outubro de 2026

Copyright (©) 2026 Erik Pimenta

# Sumário


01 Visão geral 4

1.1 Para que serve 4
1.2 O que o Turno faz 4
1.3 Ferramentas usadas 5
1.4 Identidade visual 5

02 Conceitos 7

2.1 Ciclo da tarefa 7

03 Como rodar 9

3.1 Primeira execução 9
3.2 O que sobe 9
3.3 Carga de demonstração 9
3.4 Sem Docker para as aplicações 10
3.5 Parar e limpar 10

04 Guia do painel 11

4.1 Chave de acesso 11
4.2 Visão geral 11
4.3 Tarefas 12
4.4 Detalhes da tarefa 13
4.5 Recorrentes 14
4.6 Workers 15
4.7 Fila de erros 16
4.8 Tema escuro 17

05 Escrevendo handlers 18

5.1 Estrutura 18
5.2 Registro 19
5.3 Enfileirando 19

06 Garantias de execução 20

6.1 Uma execução por tentativa 20
6.2 Servidores que caem 20
6.3 Encerramento gracioso 20
6.4 Rotinas com um único líder 21

07 Resiliência 22

7.1 Ajustes por tipo 22


Copyright (©) 2026 Erik Pimenta

08 Escala e operação 23

8.1 Mais workers 23
8.2 Filas e vagas 23
8.3 Redis com quórum 23
8.4 Azure Service Bus 23
8.5 Saúde e logs 24
8.6 Limpeza automática 24

09 API 25

9.1 Agendamento recorrente 25
9.2 Erros 26

10 Configuração 27

10.1 Variáveis do Docker 28

11 Testes e automação 29

11.1 Testes de unidade 29
11.2 Testes distribuídos 29
11.3 Integração contínua 29
11.4 Commits diários 29

12 Solução de problemas 30

Copyright (©) 2026 Erik Pimenta

# 01

# Visão geral

O Turno é um motor de tarefas em segundo plano no estilo Hangfire. Uma API recebe as tarefas, vários workers idênticos dividem as mesmas filas e um painel mostra em tempo real o que está na fila, executando, com sucesso ou com falha. Cada tarefa roda uma única vez por tentativa, mesmo com mensagens duplicadas, workers que caem no meio do processo e reinícios do broker.

## 1.1 Para que serve

Trabalho pesado não deve rodar dentro da requisição do usuário: gerar relatórios, processar arquivos, sincronizar com parceiros, enviar notificações em massa. O Turno tira esse trabalho do caminho, distribui entre quantas máquinas forem necessárias e cuida do que dá errado: novas tentativas, disjuntor, tempo limite, fila de erros e recuperação de workers que somem.

O repositório entrega a estrutura, a arquitetura, a documentação e a ferramenta pronta para rodar. Quem usa configura as conexões, escreve os próprios handlers e decide quantos workers subir.

## 1.2 O que o Turno faz

- **Fila distribuída:** workers clonados consomem as mesmas filas do RabbitMQ ou do Azure Service Bus pelo MassTransit.
- **Uma execução por vez:** lock Redlock no Redis antes de começar e troca atômica de estado no PostgreSQL com token de despacho e lease.
- **Recuperação:** se um worker some, a tarefa volta para a fila sozinha e a tentativa fica registrada como worker perdido.
- **Resiliência com Polly:** nova tentativa imediata para falhas passageiras, disjuntor por tipo, tempo limite e backoff exponencial com jitter.
- **Fila de erros:** tarefas que esgotam as tentativas vão para a tabela `dead_letters` e para a fila `turno-dead-letter` do broker.
- **Agendamento:** imediato, com atraso, em data marcada ou recorrente por cron com fuso horário.
- Chave de concorrência, idempotência, cancelamento e progresso em todas as tarefas.
- Painel em Blazor WebAssembly com atualização por SignalR, identidade visual "caixa única", tema claro e escuro e layout para celular.

Copyright (©) 2026 Erik Pimenta

## 1.3 Ferramentas usadas

| Ferramenta | Versão | Papel no projeto |
|---|---|---|
| C# e .NET | .NET 10 | Linguagem e plataforma de todos os projetos |
| Worker Service | .NET 10 | Hospedagem dos workers, serviços em segundo plano e encerramento gracioso |
| ASP.NET Core | .NET 10 | API REST com Minimal APIs, Problem Details e hospedagem do painel |
| MassTransit | 8.5.10 | Mensageria, filas por worker, outbox transacional e fila de erros |
| RabbitMQ | 4 (imagem Docker) | Broker padrão |
| Azure Service Bus | opcional | Broker alternativo, trocado só por configuração |
| Redis | 7 (imagem Docker) | Locks distribuídos |
| RedLock.net | 2.3.2 | Algoritmo Redlock com quórum sobre o StackExchange.Redis 2.13.17 |
| Polly | 8.8.0 | Nova tentativa, disjuntor e tempo limite por tipo de tarefa |
| PostgreSQL | 17 (imagem Docker) | Estado das tarefas, histórico e outbox |
| Entity Framework Core | 10.0.12 | Acesso a dados e migrações, com o provedor Npgsql 10.0.3 |
| Cronos | 0.13.0 | Expressões cron com segundos e fuso horário |
| Blazor WebAssembly | .NET 10 | Painel que roda no navegador |
| SignalR | .NET 10 | Atualização do painel em tempo real |
| OpenAPI e Scalar | .NET 10 e 2.18.0 | Especificação e documentação interativa da API em `/docs` |
| Instrument Sans, Bricolage Grotesque e IBM Plex Mono | OFL-1.1 | Tipografia do painel, incluída no projeto |
| xUnit | 2.9.3 | Testes de unidade e testes distribuídos |
| Docker Compose | v2 | Ambiente completo com um comando |
| GitHub Actions | — | Integração contínua e commits diários de atividade |

## 1.4 Identidade visual

O painel segue a identidade "caixa única", a mesma das outras aplicações do autor (como o Bento). Um bento é uma caixa só, dividida em compartimentos, e o painel segue a mesma ideia:

- **A mesa:** o fundo, o menu lateral e o cabeçalho ficam no mesmo tom claro, sem bordas. O item ativo do menu vira um compartimento branco.
- **A caixa:** os blocos de cada página dividem paredes finas de 1 px dentro de uma única caixa de cantos arredondados, em vez de cartões soltos com sombra.
- **A tampa:** o bloco principal da visão geral é o único escuro, com as execuções da última hora em número grande e as barras por minuto em branco.

Copyright (©) 2026 Erik Pimenta

- **Tipografia:** títulos e números em Bricolage Grotesque fina e condensada, textos em Instrument Sans e IDs, cron e JSON em IBM Plex Mono. Sem caixa-alta e sem rótulos em mono.
- **Cor só onde informa:** o estado aparece como um ponto colorido ao lado do texto neutro. A marca é sempre em preto e branco.

| Cor | Uso |
|---|---|
| `#F5F4F4` | Mesa: fundo das páginas, menu lateral e cabeçalho |
| `#FFFFFF` | Caixa: compartimentos, tabelas e formulários |
| `#E8E7E6` | Paredes e divisórias |
| `#BAB7B6` | Elementos desligados e estado cancelado |
| `#4D4B4B` | Textos de apoio |
| `#333232` | Tampa no tema escuro |
| `#000000` | Texto, tampa, botões principais e progresso |

| Estado | Cor | Quando aparece |
|---|---|---|
| Executando | `#000000` | Um worker está rodando a tarefa agora |
| Sucesso | `#3E6B2A` | A tarefa terminou bem |
| Falha | `#9B2F1E` | Tentativas esgotadas ou erro permanente |
| Agendada | `#7A5A12` | Espera um horário: atraso, data marcada ou nova tentativa |
| Na fila | `#2B4C5A` | Pronta, aguardando uma vaga livre |
| Cancelada | `#BAB7B6` | Cancelada antes ou durante a execução |

As cores de estado aparecem só nos marcadores; o texto continua em tinta neutra. No tema escuro os mesmos materiais ficam escuros e as cores de estado ganham versões mais claras. As fontes vêm junto com o projeto, em `wwwroot/fonts`, sem CDN.

Copyright (©) 2026 Erik Pimenta

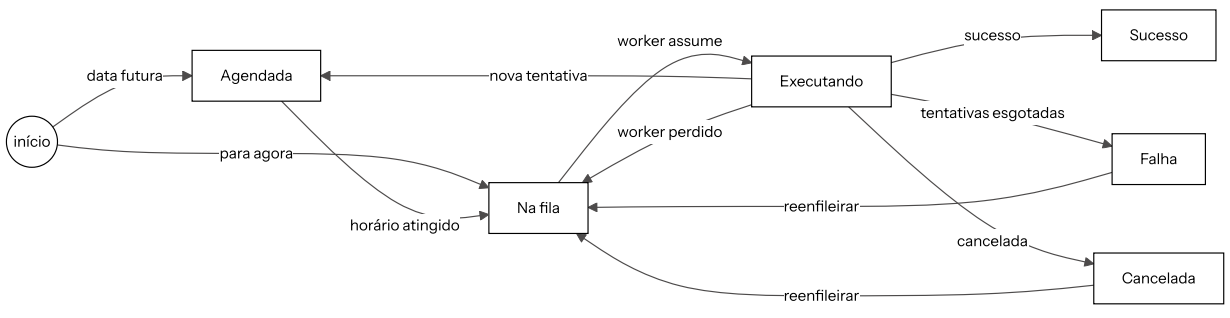
# 02

# Conceitos

Os termos abaixo aparecem no painel, na API e no restante deste manual.

| Termo | Significado |
|---|---|
| Tarefa | Um pedido de trabalho: tipo, fila, payload em JSON, limite de tentativas e tempo limite. |
| Tipo | Nome do handler que executa a tarefa, como `report.generate`. |
| Fila | Canal do broker por onde a tarefa chega aos workers. Cada worker escolhe quais filas atende e com quantas vagas. |
| Worker | Instância do `Turno.Worker`. Todos são iguais e podem ser adicionados ou removidos a qualquer momento. |
| Vaga | Uma execução simultânea dentro de um worker. A soma das vagas é a capacidade do cluster. |
| Tentativa | Cada execução de uma tarefa. Falhas passageiras geram novas tentativas até o limite. |
| Token de despacho | Identificador novo gerado sempre que a tarefa entra na fila. Só a mensagem com o token atual inicia a tarefa. |
| Lease | Prazo de posse da tarefa por um worker, renovado a cada 10 s. Se vence, outro worker pode assumir. |
| Chave de concorrência | Texto livre: tarefas com a mesma chave nunca rodam ao mesmo tempo. |
| Fila de erros | Lugar das tarefas que esgotaram as tentativas ou tiveram erro permanente. |
| Agendamento recorrente | Tarefa criada automaticamente por uma expressão cron. |

## 2.1 Ciclo da tarefa



| De | Para | Quando |
|---|---|---|
| criação | Agendada | Com atraso ou data futura |
| criação | Na fila | Para executar agora |
| Agendada | Na fila | O horário chegou |
| Na fila | Executando | Um worker assumiu a tarefa |
| Executando | Sucesso | O handler terminou sem erro |
| Executando | Agendada | Falhou e ainda há tentativas; disjuntor aberto; worker encerrado |
| Executando | Na fila | O worker parou de responder e o lease venceu |
| Executando | Falha | Tentativas esgotadas ou `PermanentJobException` |
| Executando | Cancelada | Cancelamento pedido durante a execução |
| Falha ou Cancelada | Na fila | Reenfileirada pelo painel ou pela API |

Copyright (©) 2026 Erik Pimenta

# 03

# Como rodar

O caminho mais curto é o Docker: um comando sobe banco, Redis, RabbitMQ, a API com o painel e três workers.

## 3.1 Primeira execução

1. Instale o Docker (Docker Desktop no Windows e no macOS).
2. Baixe o repositório e abra a pasta.
3. No Windows, dê dois cliques em `iniciar.cmd`. No Linux e no macOS, rode `./iniciar.sh`.
4. O script cria o `.env` a partir do `.env.example`, gera senhas aleatórias para PostgreSQL, Redis e RabbitMQ e uma chave de acesso para a API.
5. Quando a API responde, o navegador abre em `http://localhost:8080` e a chave aparece no terminal.

**Dados sensíveis.** O `.env` fica só na sua máquina e está no `.gitignore`. O repositório não traz senhas, chaves nem endereços reais.

## 3.2 O que sobe

| Serviço | Endereço | Função |
|---|---|---|
| `api` | `http://localhost:8080` | API REST, painel e hub SignalR |
| `worker` × 3 | sem porta | Workers que consomem as filas |
| `postgres` | `127.0.0.1:5432` | Estado das tarefas e outbox |
| `redis` | `127.0.0.1:6379` | Locks distribuídos |
| `rabbitmq` | `http://localhost:15672` | Broker e painel do RabbitMQ |

Todas as portas ficam presas a `127.0.0.1`. As portas e a quantidade de workers podem ser trocadas no `.env` (`TURNO_PORT`, `TURNO_WORKERS` e as portas de cada serviço).

## 3.3 Carga de demonstração

Os quatro handlers de exemplo simulam relatórios, arquivos, integrações instáveis e notificações. Para enviar um lote misto:

```
./scripts/demo.sh 60
powershell -File scripts\demo.ps1 -Count 60
```

Copyright (©) 2026 Erik Pimenta

O lote inclui tarefas imediatas, atrasadas, com falha aleatória e uma que sempre falha, para mostrar novas tentativas, disjuntor e fila de erros no painel.

## 3.4 Sem Docker para as aplicações

Com o .NET SDK 10 instalado, suba só a infraestrutura e rode API e workers pelo `dotnet`:

```
docker compose up -d postgres redis rabbitmq
```

Guarde as conexões com `dotnet user-secrets`, em `src/Turno.Api` e em `src/Turno.Worker`, para que nada fique no repositório:

```
dotnet user-secrets set "ConnectionStrings:Turno" "Host=localhost;Database=turno;Username=turno;Password=
<senha>"
dotnet user-secrets set "Turno:Messaging:RabbitMq:Username" "turno"
dotnet user-secrets set "Turno:Messaging:RabbitMq:Password" "<senha>"
dotnet user-secrets set "Turno:Redis:Connections:0" "localhost:6379,password=<senha>"
```

```
dotnet run --project src/Turno.Api
dotnet run --project src/Turno.Worker
dotnet run --project src/Turno.Worker
```

Cada `dotnet run` do worker é mais uma instância no cluster. A API aplica as migrações; os workers esperam o banco ficar pronto.

## 3.5 Parar e limpar

```
docker compose down
docker compose down -v
```

O primeiro comando para tudo e mantém os dados. O segundo apaga também os volumes do banco, do Redis e do RabbitMQ.

Copyright (©) 2026 Erik Pimenta

# 04

# Guia do painel

O painel abre na visão geral e é atualizado em tempo real pelo SignalR. O indicador no rodapé do menu lateral mostra se a conexão está ao vivo.

## 4.1 Chave de acesso

Quando `Turno:Api:ApiKey` está definida, o painel pede a chave na primeira visita. Ela fica só na memória da aba e é enviada no cabeçalho `X-Turno-Key`. Sem chave configurada, o painel abre direto.

## 4.2 Visão geral

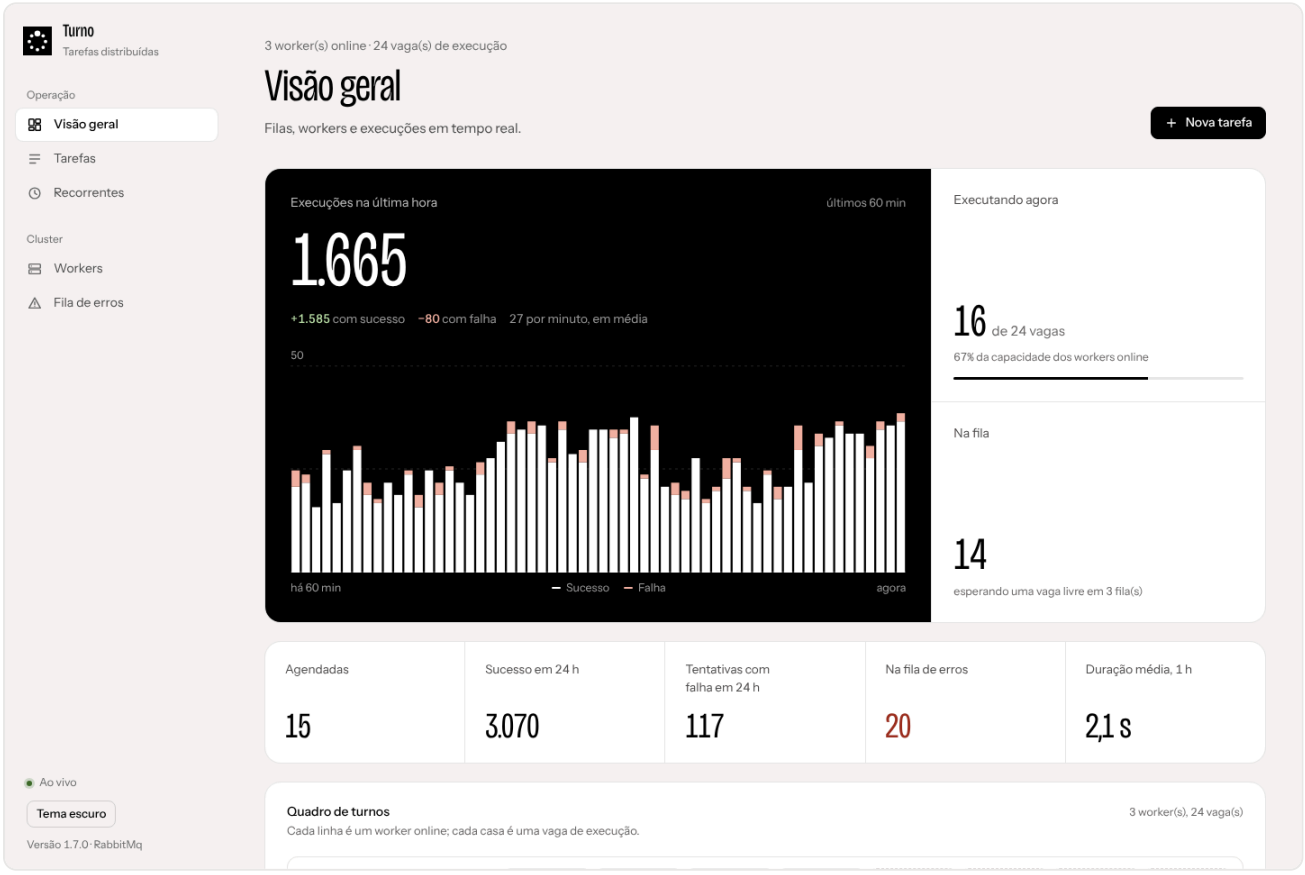



Figura 4.1 Visão geral: as execuções da última hora na tampa escura e os indicadores nos compartimentos da caixa

- **Tampa:** o bloco escuro mostra as execuções da última hora em número grande, os sucessos (+) e as falhas (−) e as execuções por minuto em barras, com as falhas em destaque.
- **Executando agora e na fila:** ao lado da tampa, a ocupação das vagas sobre a capacidade total dos workers online e as tarefas esperando vaga.
- **Faixa de indicadores:** agendadas, sucessos e tentativas com falha em 24 h, fila de erros e duração média da última hora. Cada indicador abre a lista já filtrada.

Copyright (©) 2026 Erik Pimenta

- **Quadro de turnos:** cada linha é um worker online e cada casa uma vaga. Casas ocupadas mostram o tipo da tarefa, o tempo decorrido e o progresso; um clique abre os detalhes.
- **Filas e falhas recentes:** tarefas pendentes por fila e as últimas entradas da fila de erros.

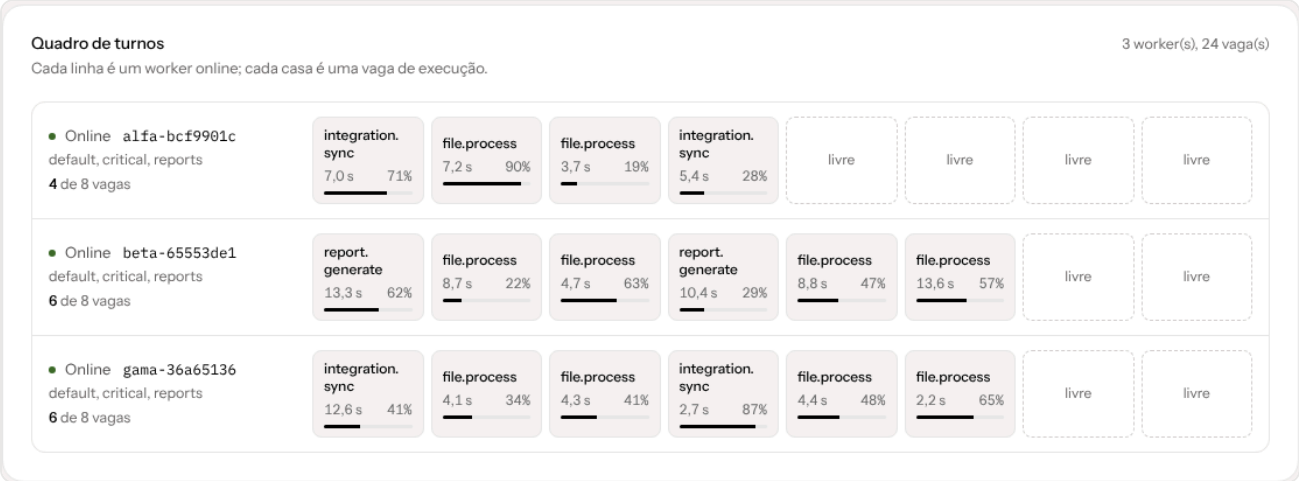



Figura 4.2 Quadro de turnos: três workers online, com as vagas ocupadas e livres de cada um

## 4.3 Tarefas

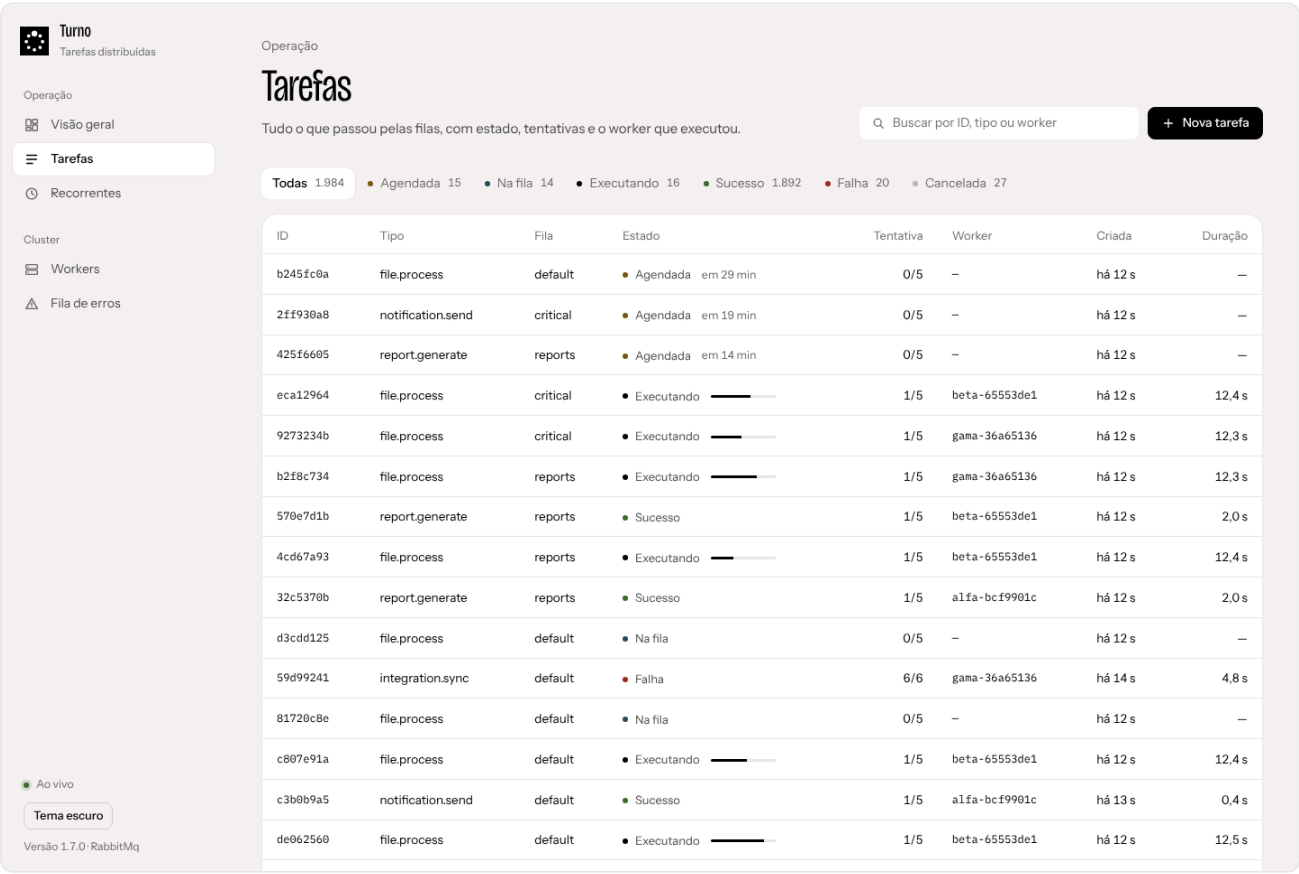



Figura 4.3 Lista de tarefas com filtro por estado e busca

Lista paginada com filtro por estado, busca por tipo, ID ou worker e colunas de fila, tentativa, worker, criação e duração. O botão **Nova tarefa** abre um formulário com tipo (com os tipos anunciados pelos workers), fila, payload em JSON, quando executar (agora, daqui a alguns segundos ou em data e hora), máximo de tentativas, tempo limite e chave de concorrência.

Copyright (©) 2026 Erik Pimenta

## 4.4 Detalhes da tarefa

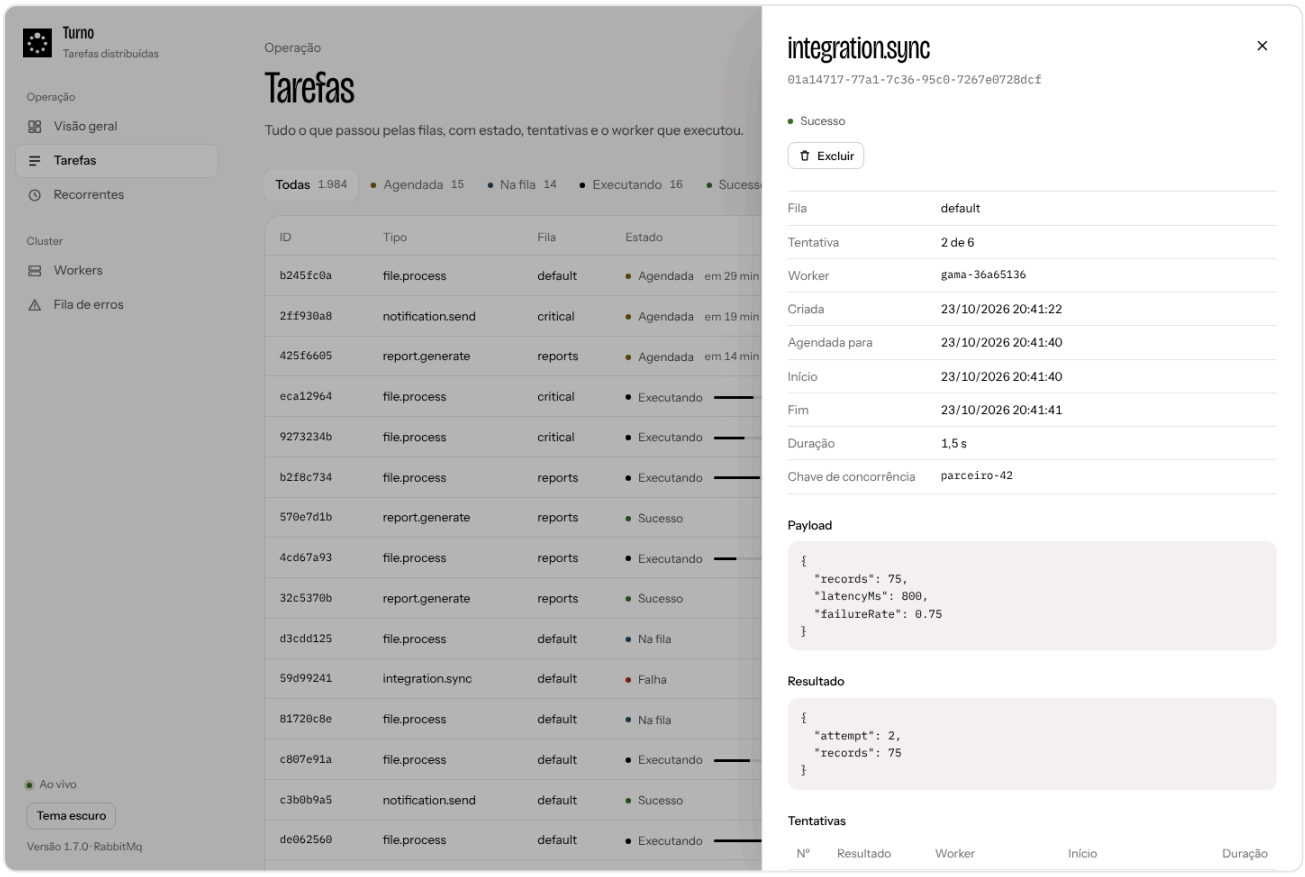



**Figura 4.4** Detalhes com tentativas, linha do tempo, payload e resultado

O painel lateral mostra o estado, as tentativas com worker, duração e erro, a linha do tempo de mudanças de estado, o payload e o resultado. Dali é possível cancelar, reenfileirar ou excluir, conforme o estado.

Copyright (©) 2026 Erik Pimenta

## 4.5 Recorrentes

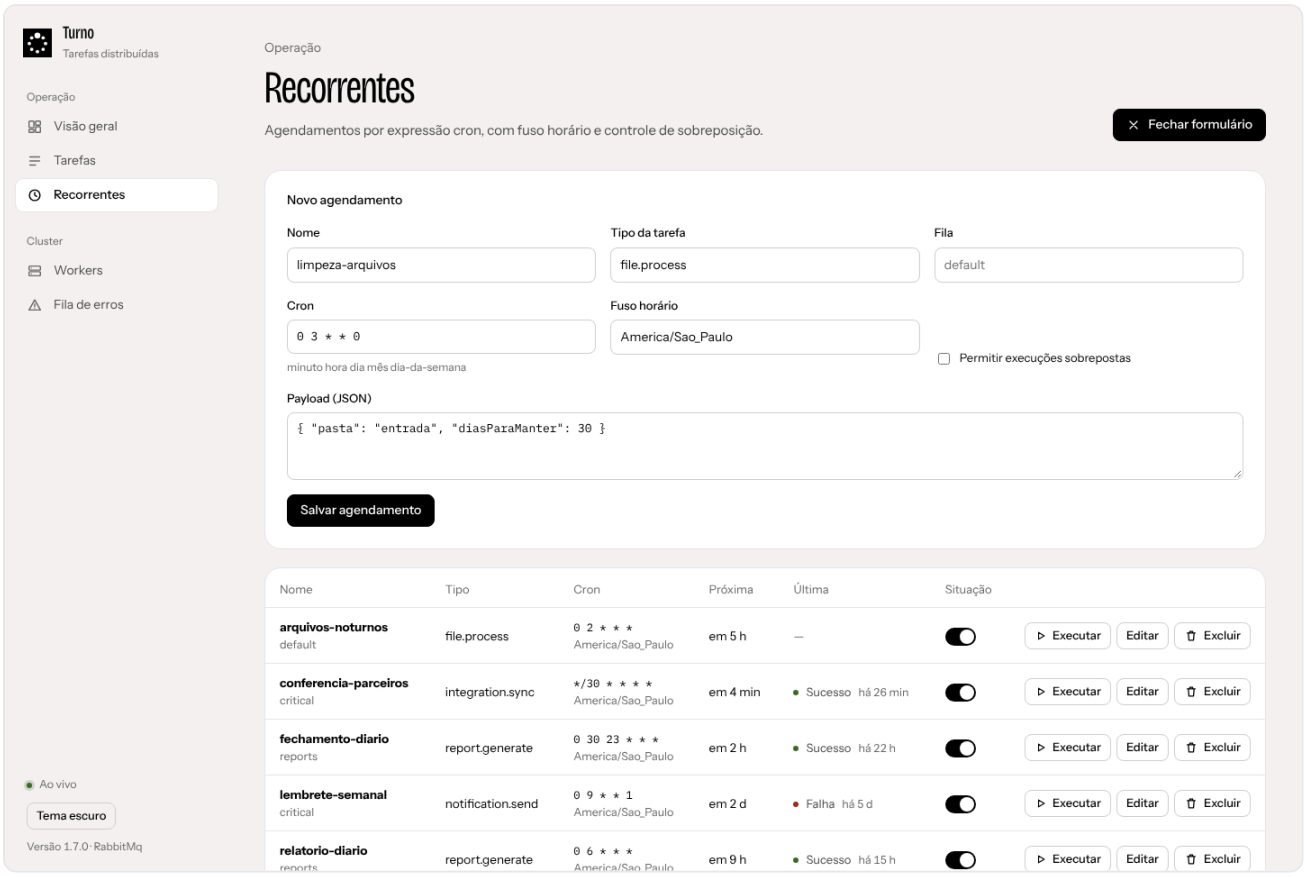



**Figura 4.5** Agendamentos recorrentes com o formulário aberto e a próxima execução de cada um

Cria e edita agendamentos por cron (5 ou 6 campos) com fuso horário, fila, payload e se pode haver sobreposição. Cada linha pode ser disparada na hora, pausada, reativada ou excluída. Agendamentos também podem ser declarados em `Turno:RecurringJobs` no appsettings.

Copyright (©) 2026 Erik Pimenta

## 4.6 Workers

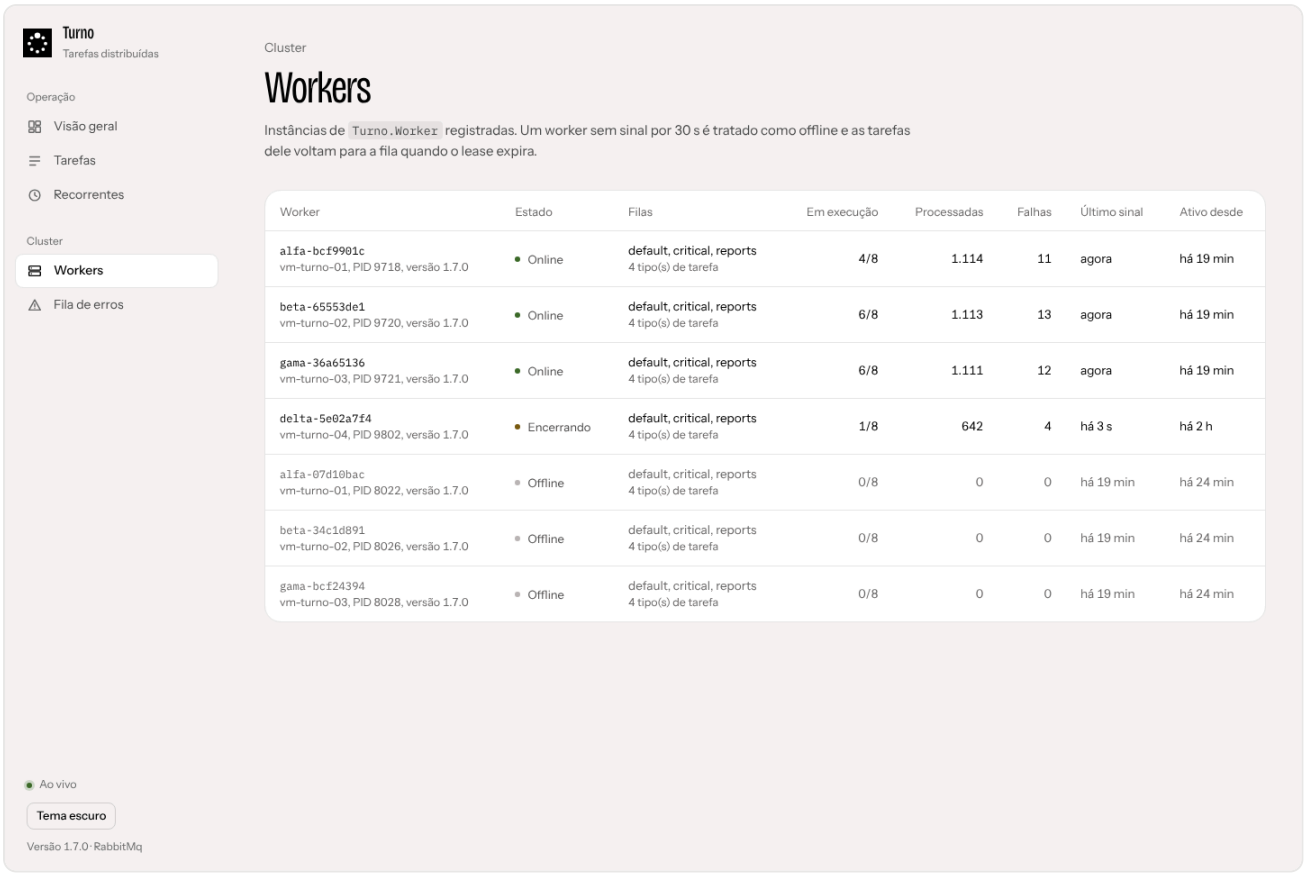



**Figura 4.6** Workers registrados com filas, carga e último sinal de vida

Todas as instâncias registradas, com filas, vagas em uso, tarefas processadas e com falha, horário de início e último sinal de vida. Um worker que para de enviar sinal por 30 s aparece como offline e sai da lista depois de um dia.

Copyright (©) 2026 Erik Pimenta

## 4.7 Fila de erros

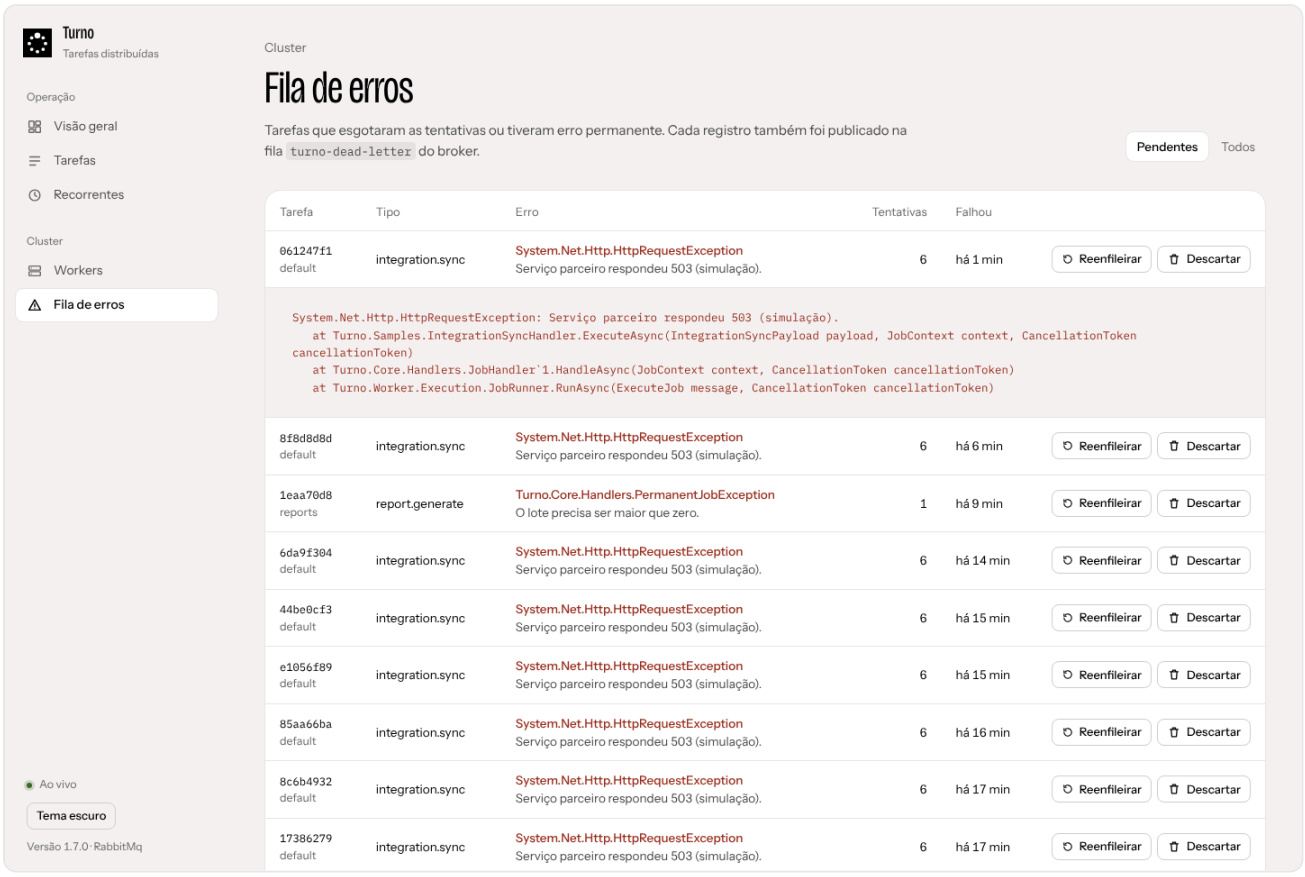



**Figura 4.7** Fila de erros com o erro de cada tarefa e o stack trace aberto

Tarefas que esgotaram as tentativas ou tiveram erro permanente, com tipo do erro, mensagem e stack trace. **Reenfileirar** devolve a tarefa original à fila com novas tentativas; **Descartar** remove o registro.

Copyright (©) 2026 Erik Pimenta

## 4.8 Tema escuro

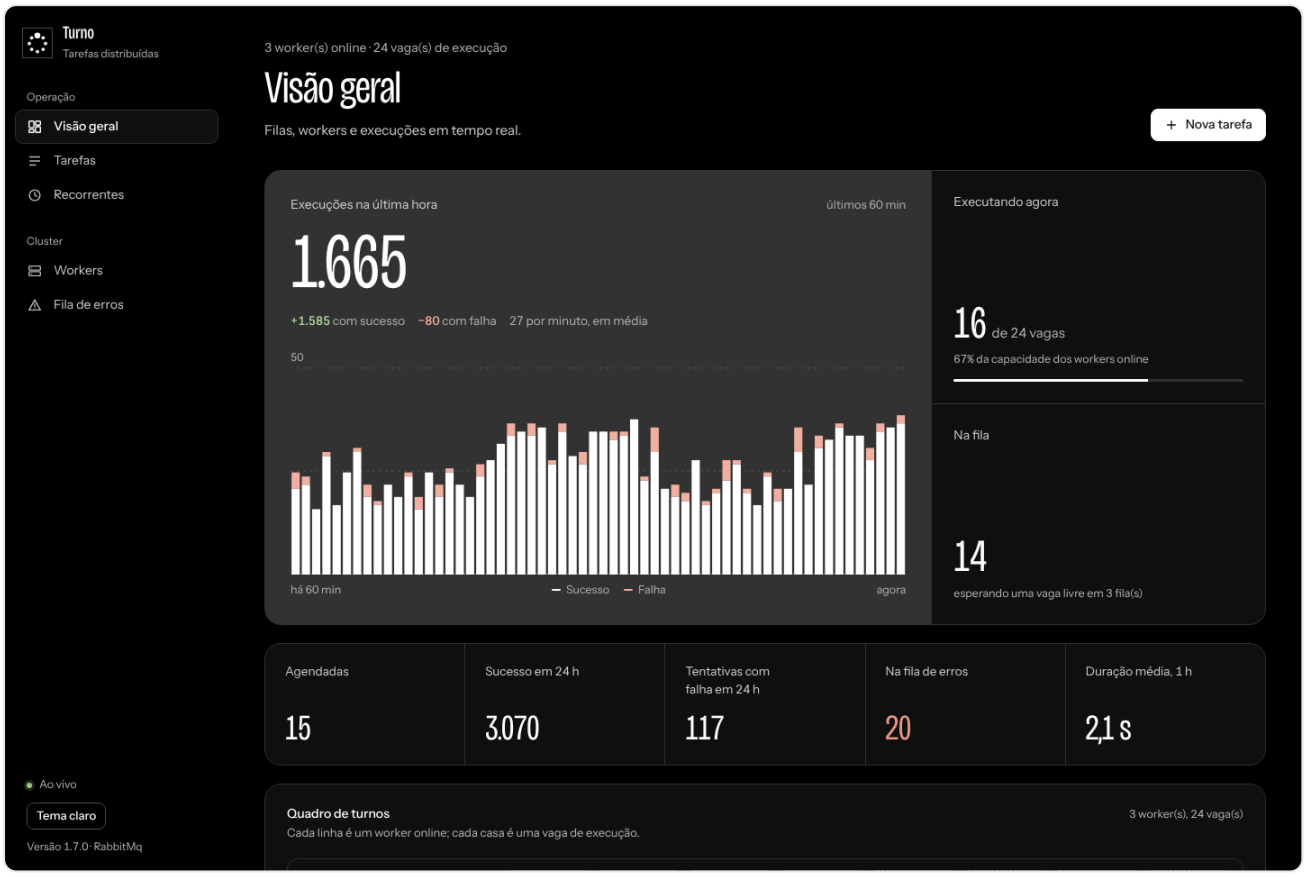



Figura 4.8 Visão geral no tema escuro

O botão no menu lateral alterna entre claro e escuro; a escolha fica guardada no navegador. Sem escolha, o painel segue o tema do sistema.

Copyright (©) 2026 Erik Pimenta

# 05

# Escrevendo handlers

Um handler é uma classe que executa um tipo de tarefa. O Turno cuida de fila, lock, tentativas, progresso e cancelamento; o handler só faz o trabalho.

## 5.1 Estrutura

```
using Turno.Core.Handlers;

namespace MinhaEmpresa.Tarefas;

public sealed record GerarBoletosPayload(int Lote, string Competencia);

[JobType("boletos.gerar")]
public sealed class GerarBoletosHandler(IBoletoService boletos) : JobHandler<GerarBoletosPayload>
{
    protected override async Task ExecuteAsync(GerarBoletosPayload payload, JobContext context,
CancellationToken cancellationToken)
    {
        if (payload.Lote <= 0)
        {
            throw new PermanentJobException("O lote precisa ser maior que zero.");
        }

        var total = await boletos.ContarAsync(payload.Lote, cancellationToken);
        for (var i = 0; i < total; i++)
        {
            await boletos.GerarAsync(payload.Lote, i, cancellationToken);
            await context.ReportProgressAsync((i + 1) * 100 / total, cancellationToken);
        }

        context.SetResult(new { total });
    }
}
```

| Recurso | Como usar |
|---|---|
| Nome do tipo | `[JobType("...")]` com letras minúsculas, números, ponto, hífen ou sublinhado |
| Payload | Chega tipado e desserializado do JSON em camelCase; payload inválido vira erro permanente |
| Progresso | `context.ReportProgressAsync(0 a 100)`, enviado ao painel no máximo uma vez por segundo |
| Resultado | `context.SetResult(objeto)`, gravado em JSON (até 32 KB) |
| Cancelamento | O `CancellationToken` dispara em cancelamento, tempo limite, perda do lease e encerramento do worker |
| Erro passageiro | Qualquer exceção comum: o Polly tenta de novo e, depois, a tarefa volta com backoff |
| Erro permanente | `PermanentJobException`: vai direto para a fila de erros, sem novas tentativas |
| Dependências | Injetadas pelo construtor, com escopo próprio por execução |

Copyright (©) 2026 Erik Pimenta

## 5.2 Registro

```
builder.Services.AddJobHandlersFromAssembly(typeof(GerarBoletosHandler).Assembly);
```

Coloque essa linha no `Program.cs` do worker. Os tipos registrados são anunciados pelo worker e aparecem no formulário de nova tarefa. Os handlers de exemplo podem ser desligados com `Turno:Samples:Enabled=false`.

## 5.3 Enfileirando

```
curl -X POST http://localhost:8080/api/jobs \
  -H "Content-Type: application/json" \
  -H "X-Turno-Key: <sua chave>" \
  -H "Idempotency-Key: boletos-2026-10-lote-7" \
  -d '{"type":"boletos.gerar","queue":"default","payload":{"lote":7,"competencia":"2026-10"}}'
```

| Campo | Uso |
|---|---|
| `type` | Obrigatório. Tipo do handler |
| `queue` | Fila; `default` quando omitida |
| `payload` | Qualquer JSON, até 256 KB |
| `runAt` ou `delaySeconds` | Executa numa data e hora ou depois de um atraso |
| `maxAttempts` | Limite de tentativas (1 a 100); padrão da configuração quando omitido |
| `timeoutSeconds` | Tempo limite da execução |
| `idempotencyKey` | Evita duplicatas; também aceito no cabeçalho `Idempotency-Key` |
| `concurrencyKey` | Impede execuções simultâneas com a mesma chave |

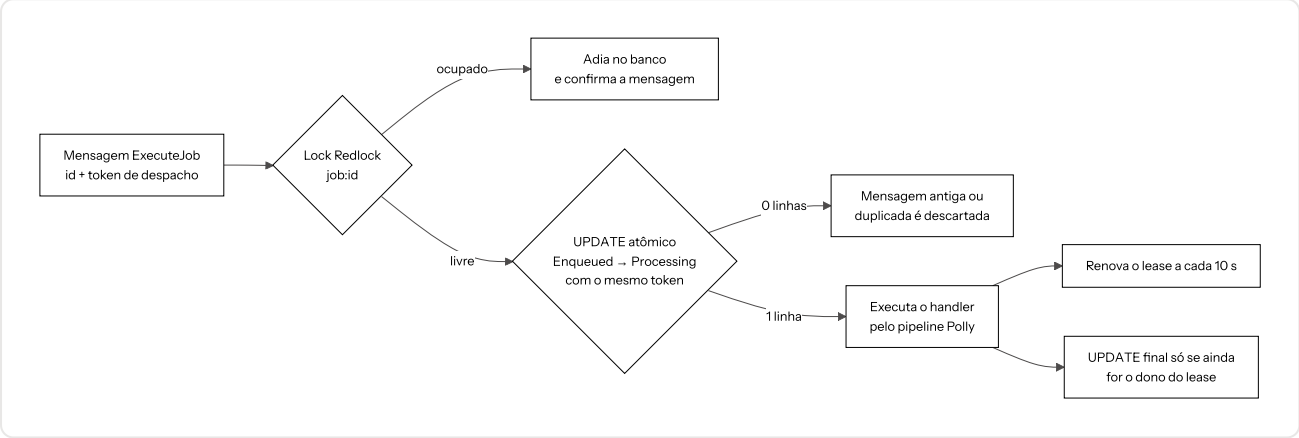
Copyright (©) 2026 Erik Pimenta

# 06

# Garantias de execução

Como o Turno impede que duas instâncias rodem a mesma tarefa e o que acontece quando um servidor cai no meio do processo.

## 6.1 Uma execução por tentativa




1. **Token de despacho.** Toda vez que a tarefa entra na fila recebe um `dispatch_id` novo, que vai na mensagem. Reenvios, duplicatas e mensagens antigas carregam um token vencido e são descartados.
2. **Redlock.** O worker pega o lock `job:{id}` no Redis antes de tocar no banco. Com várias instâncias de Redis, o lock exige quórum. Se o lock está ocupado, a tarefa é adiada no banco e a mensagem é confirmada, sem perder trabalho.
3. **Troca atômica.** A passagem para executando é um único `UPDATE ... WHERE state = 'Enqueued' AND dispatch_id = @token`. Só uma instância consegue a linha.
4. **Lease com cerca.** O término só é gravado se o worker ainda for o dono ( `worker_id` , `dispatch_id` e estado iguais). Um worker que travou e voltou não sobrescreve quem assumiu.
5. **Outbox transacional.** A mensagem para o broker é gravada na mesma transação da tarefa e entregue depois do commit. Se o banco recusou, nada é publicado.

## 6.2 Servidores que caem

Cada execução renova o lease a cada 10 s. O serviço de recuperação procura leases vencidos (60 s mais 15 s de tolerância), marca a tentativa como worker perdido e devolve a tarefa à fila com um token novo. Se o worker antigo voltar, a cerca descarta o resultado dele.

## 6.3 Encerramento gracioso

Ao receber o sinal de parada, o worker deixa de pegar tarefas e espera até 30 s pelas que estão rodando. As que não terminam são canceladas e voltam à fila sem contar tentativa. No Docker, o `stop_grace_period` é de 45 s para caber esse prazo.

Copyright (©) 2026 Erik Pimenta

## 6.4 Rotinas com um único líder

O agendador (tarefas agendadas e recorrentes), a recuperação e a limpeza rodam em todos os workers, mas a cada ciclo só um deles, eleito por um lock Redlock, executa a rotina. Dá para ligar quantos workers quiser sem duplicar esse trabalho.

Copyright (©) 2026 Erik Pimenta

# 07

# Resiliência

Cada tipo de tarefa tem o próprio pipeline do Polly, montado na primeira execução e reaproveitado depois.

| Camada | Ferramenta | Padrão |
|---|---|---|
| Tempo limite | Polly `Timeout` | 10 min, ajustável por tipo ou por tarefa |
| Nova tentativa imediata | Polly `Retry` | 2 novas tentativas a partir de 500 ms, só para erros passageiros |
| Disjuntor | Polly `CircuitBreaker` | Abre com 50% de falhas em 30 s (mínimo de 10 execuções) e fica aberto 30 s; as tarefas são adiadas sem contar tentativa |
| Novas tentativas da tarefa | Agendador do Turno | Até 5 tentativas, com espera de 15 s, 30 s, 60 s… até 15 min e 20% de jitter |
| Fila de erros | `dead_letters` e `turno-dead-letter` | Recebe a tarefa quando as tentativas acabam ou em erro permanente |
| Mensagens envenenadas | MassTransit `_error` | Mensagens que não conseguem nem ser processadas |

## 7.1 Ajustes por tipo

```
"Turno": {
  "Resilience": {
    "DefaultMaxAttempts": 5,
    "BackoffBase": "00:00:15",
    "BackoffMax": "00:15:00",
    "Types": {
      "report.generate": { "Timeout": "00:30:00" },
      "integration.sync": {
        "ImmediateRetries": 3,
        "MaxAttempts": 6,
        "CircuitBreaker": { "FailureRatio": 0.3, "BreakDuration": "00:01:00" }
      }
    }
  }
}
```

**Dica.** Lance `PermanentJobException` para erros que nunca vão se resolver sozinhos (dados inválidos, registro inexistente). Assim a tarefa não gasta tentativas nem segura vagas.

Copyright (©) 2026 Erik Pimenta

# 08

# Escala e operação

Os workers não guardam estado: qualquer um pode morrer, reiniciar ou ser adicionado a qualquer momento.

## 8.1 Mais workers

```
docker compose up -d --scale worker=8
```

Fora do Docker, basta iniciar mais processos `Turno.Worker` apontando para o mesmo banco, Redis e broker, em quantas máquinas quiser.

## 8.2 Filas e vagas

```
"Turno": {
  "Worker": {
    "Queues": [
      { "Name": "default", "Concurrency": 4 },
      { "Name": "critical", "Concurrency": 2 },
      { "Name": "reports", "Concurrency": 2 }
    ]
  }
}
```

Workers diferentes podem atender filas diferentes: por exemplo, máquinas maiores só com `reports`. A fila no broker se chama `turno-<fila>`; o prefixo muda em `Turno:Messaging:QueuePrefix`.

## 8.3 Redis com quórum

Para desenvolvimento, uma conexão basta. Em produção, use 3 ou 5 instâncias independentes de Redis em `Turno:Redis:Connections`: o lock só é concedido com a maioria, como no Redlock original. Sem quórum, o worker adia a tarefa em vez de arriscar.

## 8.4 Azure Service Bus

```
Turno__Messaging__Transport=AzureServiceBus
Turno__Messaging__AzureServiceBus__ConnectionString=<connection string do namespace>
```

Nada mais muda: filas, fila de erros e eventos do painel funcionam igual.

Copyright (©) 2026 Erik Pimenta

## 8.5 Saúde e logs

| Rota | O que verifica |
|---|---|
| `/health/live` | Processo da API no ar |
| `/health/ready` | PostgreSQL, quórum do Redis e conexão com o broker |

Os logs saem no console em uma linha por evento, em português, com o ID da tarefa e o worker. No Docker: `docker compose logs -f worker`.

## 8.6 Limpeza automática

Tarefas com sucesso e canceladas são removidas depois de 7 dias, as com falha depois de 30 e os workers offline depois de 1 dia. Os prazos ficam em `Turno:Retention`.

Copyright (©) 2026 Erik Pimenta

# 09

# API

Documentação interativa em `http://localhost:8080/docs` e especificação OpenAPI em `/openapi/v1.json`.

Com `Turno:Api:ApiKey` definida, todas as rotas, menos `/api/info` e `/health`, exigem o cabeçalho `X-Turno-Key`.

| Método | Rota | Função |
|---|---|---|
| POST | `/api/jobs` | Enfileira ou agenda uma tarefa |
| POST | `/api/jobs/batch` | Enfileira até 1000 tarefas |
| GET | `/api/jobs` | Lista com filtros (`state`, `type`, `queue`, `search`) e paginação |
| GET | `/api/jobs/{id}` | Detalhes, tentativas e linha do tempo |
| POST | `/api/jobs/{id}/cancel` | Cancela uma tarefa agendada, na fila ou em execução |
| POST | `/api/jobs/{id}/requeue` | Devolve à fila uma tarefa com falha ou cancelada |
| DELETE | `/api/jobs/{id}` | Exclui uma tarefa finalizada |
| GET | `/api/stats` | Indicadores, vazão por minuto e filas |
| GET | `/api/workers` | Workers registrados |
| GET | `/api/job-types` | Tipos anunciados pelos workers online |
| GET · PUT | `/api/recurring-jobs` | Lista; cria ou atualiza pelo nome |
| POST | `/api/recurring-jobs/{id}/trigger` | Dispara agora |
| POST | `/api/recurring-jobs/{id}/enable · /disable` | Ativa ou pausa |
| DELETE | `/api/recurring-jobs/{id}` | Exclui o agendamento |
| GET | `/api/dead-letters` | Fila de erros (`pending=true` para só as pendentes) |
| POST | `/api/dead-letters/{id}/requeue` | Reenfileira a tarefa original |
| DELETE | `/api/dead-letters/{id}` | Descarta o registro |
| WS | `/hubs/jobs` | Hub SignalR com `JobsChanged` e `WorkersChanged` |

## 9.1 Agendamento recorrente

```
curl -X PUT http://localhost:8080/api/recurring-jobs \
  -H "Content-Type: application/json" \
  -H "X-Turno-Key: <sua chave>" \
```

Copyright (©) 2026 Erik Pimenta

```
-d '{"name":"fechamento-diario","type":"report.generate","queue":"reports","cron":"0 30 23 * *
*","timeZone":"America/Sao_Paulo","payload":{"rows":250000}}'
```

Com 6 campos, o primeiro é o segundo. Sem `allowConcurrent`, uma nova execução não é criada enquanto a anterior ainda estiver agendada, na fila ou executando.

## 9.2 Erros

Os erros seguem o padrão Problem Details (RFC 9457), com mensagens em português e os campos inválidos em `errors`:

```
{ "title": "Dados inválidos para a tarefa.", "status": 400, "errors": { "type": ["Informe o tipo da tarefa com
letras minúsculas, números, ponto, hífen ou sublinhado (até 200 caracteres)."] } }
```

Copyright (©) 2026 Erik Pimenta

# 10

# Configuração

As opções ficam no `appsettings.json` da API e do worker e podem ser sobrescritas por variáveis de ambiente, trocando `:` por `__`.

| Seção | Principais opções | Padrão |
|---|---|---|
| `ConnectionStrings:Turno` | Conexão com o PostgreSQL | obrigatória |
| `Turno:Database` | `ApplyMigrations` | ligado na API, desligado nos workers do Docker |
| `Turno:Messaging` | `Transport`, `QueuePrefix`, credenciais do RabbitMQ ou connection string do Azure Service Bus | `RabbitMq`, `turno` |
| `Turno:Redis` | `Connections`, `KeyPrefix` | `localhost:6379`, `turno` |
| `Turno:Worker` | `Name`, `Queues`, `LeaseDuration`, `LeaseRenewalInterval`, `LockExpiry`, `HeartbeatInterval`, `ShutdownTimeout` | nome da máquina; 60 s, 10 s, 30 s, 5 s, 30 s |
| `Turno:Scheduler` | `Interval`, `BatchSize` | 2 s, 200 |
| `Turno:Recovery` | `Interval`, `LeaseGracePeriod`, `RedispatchEnqueuedAfter` | 10 s, 15 s, desligado |
| `Turno:Retention` | Prazos de guarda e de worker offline | 7 d, 7 d, 30 d, 1 d |
| `Turno:Resilience` | Tentativas, backoff, tempo limite, disjuntor e `Types` | veja o capítulo Resiliência |
| `Turno:Api` | `ApiKey`, `AllowedOrigins`, `EnableApiReference` | sem chave, sem CORS, docs ligadas |
| `Turno:RecurringJobs` | Agendamentos declarados na configuração | 2 exemplos |
| `Turno:Samples` | `Enabled` | ligado |

Copyright (©) 2026 Erik Pimenta

## 10.1 Variáveis do Docker

| Variável | Uso |
|---|---|
| `TURNO_PORT` | Porta do painel e da API (8080) |
| `TURNO_WORKERS` | Quantidade inicial de workers (3) |
| `TURNO_API_KEY` | Chave de acesso; vazia desliga a exigência |
| `TURNO_POSTGRES_*` | Banco, usuário, senha e porta do PostgreSQL |
| `TURNO_RABBITMQ_*` | Usuário, senha e portas do RabbitMQ |
| `TURNO_REDIS_*` | Senha e porta do Redis |
| `TZ` | Fuso dos containers (`America/Sao_Paulo`) |

Copyright (©) 2026 Erik Pimenta

# 11

# Testes e automação

Duas suítes de testes, integração contínua e o workflow de atividade diária.

## 11.1 Testes de unidade

```
dotnet test tests/Turno.Tests
```

Cobrem máquina de estados, cron, backoff, validação das requisições, handlers, pipeline do Polly e o modelo do banco, incluindo a checagem de que não há mudança no modelo sem migração.

## 11.2 Testes distribuídos

```
docker compose up -d postgres redis rabbitmq
export TURNO_TEST_POSTGRES="Host=localhost;Username=turno;Password=<senha>;Database=postgres"
export TURNO_TEST_REDIS="localhost:6379,password=<senha>"
export TURNO_TEST_RABBITMQ="amqp://turno:<senha>@localhost:5672"
dotnet test tests/Turno.IntegrationTests
```

Cada teste sobe um cluster com vários workers contra serviços reais, num banco e num prefixo de filas próprios, e prova: 60 tarefas em 3 workers rodam uma vez cada; duplicatas e mensagens antigas são ignoradas; um worker travado tem a tarefa recuperada e o resultado descartado; tentativas esgotadas vão para a fila de erros; erro permanente pula as tentativas; a chave de concorrência impede sobreposição; cancelamento, encerramento gracioso, agendamento, recorrência e idempotência funcionam.

## 11.3 Integração contínua

O workflow `ci.yml` compila a solução, roda as duas suítes com PostgreSQL, Redis e RabbitMQ como serviços e gera as duas imagens Docker a cada push na `main`.

## 11.4 Commits diários

| Turno | Horário principal | Reserva | Brasília |
|---|---|---|---|
| 1º turno | 04:23 UTC | 05:41 UTC | 01:23 e 02:41 |
| 2º turno | 07:23 UTC | 08:41 UTC | 04:23 e 05:41 |

O workflow `diario.yml` registra a atividade em `.github/atividade.txt`. Os horários ficam na janela de menor carga do GitHub Actions, longe da virada de hora e sem coincidir com os outros repositórios do autor; o horário reserva só faz commit se o principal não rodou. Crie os secrets `GIT_NAME` e `GIT_EMAIL` em Settings › Secrets and variables › Actions.

Copyright (©) 2026 Erik Pimenta

12

# Solução de problemas

Os sintomas mais comuns, a causa provável e o que fazer.

| Sintoma | Causa provável | O que fazer |
|---|---|---|
| O painel não abre | Containers ainda subindo | `docker compose ps` e `docker compose logs api` |
| O painel pede a chave | `TURNO_API_KEY` definida | Use a chave mostrada pelo script ou a do `.env` |
| "Sem conexão em tempo real" | API reiniciando ou proxy sem WebSocket | O painel reconecta sozinho; libere WebSocket no proxy |
| Tarefas ficam na fila | Nenhum worker atende aquela fila ou não há vagas | Confira as filas em Workers e o `Turno:Worker:Queues` |
| Tarefas adiadas com "disjuntor aberto" | Muitas falhas seguidas do mesmo tipo | Corrija a dependência; o disjuntor fecha sozinho |
| Workers esperando o banco | Migrações ainda não aplicadas | Suba a API, que aplica as migrações, ou ligue `ApplyMigrations` em um worker |
| Redis sem quórum | Instâncias de Redis fora do ar | Verifique `/health/ready`; as tarefas são adiadas até o quórum voltar |
| Porta ocupada | Outro serviço usa a mesma porta | Troque a porta no `.env` e rode o script de novo |

**Recomeçar do zero.** `docker compose down -v` apaga banco, filas e locks. Depois, rode o script de início de novo.

Copyright (©) 2026 Erik Pimenta